

AHB LITE MASTER

Using SystemVerilog

Marium Waleed Mostafa

July 2026

[GitHub Repo](#)

Contents

1	Design Architecture	2
2	Assumptions and Supported Features	3
2.1	Supported Features	3
2.2	Design Assumptions	3
3	Master Interface	4
3.1	Global Signals	4
3.2	AHB-Lite Side Signals (Master to Bus)	4
3.3	Processor Side Signals	5
4	Finite State Machine	6
4.1	States	6
4.2	Transition Conditions	6
5	RTL Code	8
6	Testbench Code	12
7	Simulation Results	20
7.1	Terminal Execution Logs	20
7.2	Test 1: Byte Burst Transfer	21
7.3	Test 2: Halfword Burst Transfer	21
7.4	Test 3: Word Burst Transfer	22
7.5	Test 4: Master Busy State Handling	22
7.6	Test 5: Slave Wait-State Handling	23
7.7	Test 6: Read Data Return	23
7.8	Test 7: Slave Error Response Handling	23

1 Design Architecture

The Advanced Microcontroller Bus Architecture (AMBA) AHB-Lite protocol is an ARM open-standard, on-chip bus interface intended for high-performance, high clock-frequency, synthesizable designs[cite: 1]. It is a simplified subset of the full AMBA AHB protocol that supports only a single bus master, which removes the need for arbitration logic while retaining the pipelined, high-bandwidth nature of the original AHB bus[cite: 1].

An AHB-Lite system is generally composed of four types of components:

- **Master** – initiates read and write transfers by driving the address and control signals[cite: 1].
- **Slave** – responds to a transfer that has been selected for it, signalling success, failure, or the need for additional wait cycles[cite: 1].
- **Decoder** – decodes the address bus to generate the slave select signals (HSEL_x) and the multiplexor control[cite: 1].
- **Multiplexor** – routes the read data and response signals from the selected slave back to the master[cite: 1].

Every AHB-Lite transfer is split into two pipelined phases: an **address phase**, lasting a single clock cycle, and a **data phase**, which can be extended using the HREADY signal whenever a slave needs additional time[cite: 1]. Because the address phase of a new transfer overlaps with the data phase of the previous one, the bus can sustain high-throughput operation while still giving slaves adequate time to respond[cite: 1]. The transfer outcome is reported through HRESP, which indicates whether the transfer completed successfully (OKAY) or with an error (ERROR)[cite: 1].

The block-level structure of a typical single-master AHB-Lite system – showing the master, the address decoder, the slaves, and the read-data multiplexor – is shown in Figure 1.

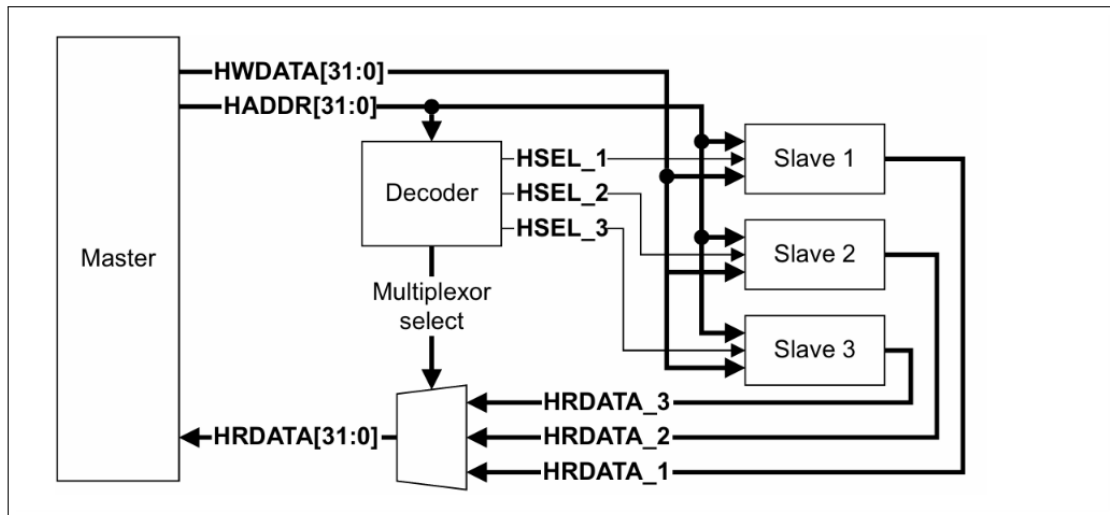


Figure 1: AHB-Lite block diagram.

2 Assumptions and Supported Features

2.1 Supported Features

- Only **SINGLE** and **INCR** (incrementing, undefined length) burst transfers are supported[cite: 1]. Fixed-length bursts (WRAP4/8/16, INCR4/8/16) are not implemented[cite: 1].
- **Transfer size** is fully supported through the HSIZE signal, driven directly from the processor-side SIZE input (byte, halfword, word, etc.)[cite: 1].
- **Locked transfers** are not supported – HMASTLOCK is tied LOW at all times[cite: 1].
- **Protection control** is not supported – HPROT is tied to a fixed, safe default value[cite: 1].

2.2 Design Assumptions

- HBURST is fixed to 3'b001 (**INCR**), since only single transfers and undefined-length incrementing bursts are supported[cite: 1]. A single transfer is therefore treated, per the AHB-Lite specification, as an incrementing burst of length one[cite: 1].
- HPROT is tied to 4'b0011, corresponding to a non-cacheable, non-bufferable, privileged, data access, which is the value recommended by the AHB-Lite specification for masters that cannot generate accurate protection information[cite: 1].
- HMASTLOCK is tied to 1'b0 since locked transfers are not required by this design[cite: 1].
- The processor-side interface uses a simple VALID/MID handshake instead of raw HTRANS encoding: VALID marks a new request as valid and MID marks whether the current beat is part of an ongoing incrementing burst[cite: 1].
- The next beat address for an ongoing burst is calculated internally by the master as $HADDR + (1 \ll SIZE)$, i.e. the address is incremented by the number of bytes indicated by HSIZE at the time the burst started[cite: 1].
- ACCEPT is asserted combinatorially whenever HREADY is HIGH, HRESP is LOW (OKAY), and VALID is HIGH, indicating that the processor's current request has been latched by the master[cite: 1].
- RVALID is asserted whenever the FSM is in the NONSEQ or SEQ state and HREADY is HIGH, indicating that valid read data is available on RDATA[cite: 1].
- An HRESP ERROR response returns the FSM directly to the IDLE state, and the error/status is forwarded to the processor through the RESP signal[cite: 1].

3 Master Interface

The AHB-Lite master module sits between the processor (or any generic request source) and the AHB-Lite bus fabric (decoder / multiplexor / slaves)[cite: 1]. It translates simple processor-side request/response handshaking into AHB-Lite compliant address, control, and data signalling[cite: 1]. The overall interface is shown in Figure 3.

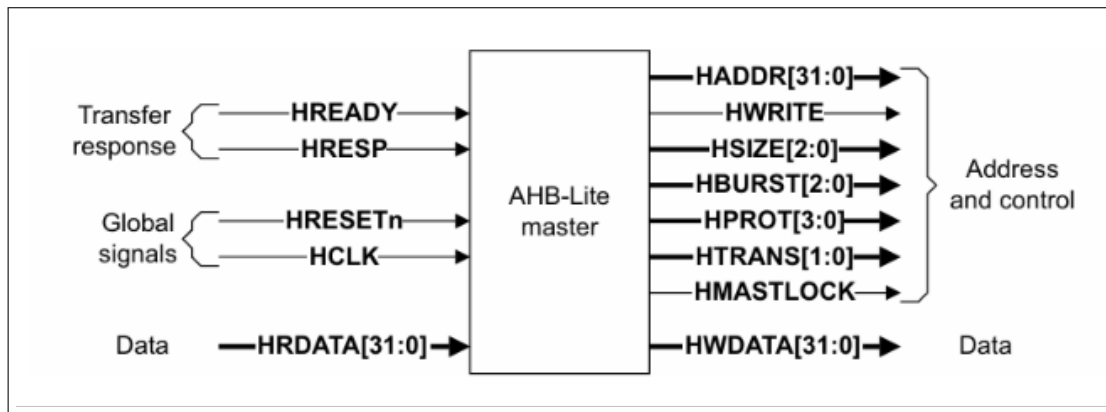


Figure 2: AHB-Lite master interface.

3.1 Global Signals

Signal	Dir.	Description
HCLK	In	Bus clock. All signals are timed to its rising edge[cite: 1].
HRESETn	In	Active-LOW asynchronous reset for the whole master[cite: 1].

3.2 AHB-Lite Side Signals (Master to Bus)

Signal	Dir.	Description
HADDR[31:0]	Out	Address of the current transfer[cite: 1].
HWRITE	Out	HIGH for a write transfer, LOW for a read[cite: 1].
HSIZE[2:0]	Out	Size of the transfer (byte / halfword / word / ...)[cite: 1].
HBURST[2:0]	Out	Burst type; fixed to INCR in this design[cite: 1].
HTRANS[1:0]	Out	Transfer type: IDLE, BUSY, NONSEQ, SEQ[cite: 1].
HWDATA[31:0]	Out	Write data bus[cite: 1].
HPROT[3:0]	Out	Protection control; tied to a fixed value (unused)[cite: 1].
HMASTLOCK	Out	Locked-transfer indicator; tied LOW (unused)[cite: 1].
HREADY	In	HIGH when the previous transfer has completed[cite: 1].
HRESP	In	Transfer response: LOW = OKAY, HIGH = ERROR[cite: 1].
HRDATA[31:0]	In	Read data returned from the selected slave (via the mux)[cite: 1].

3.3 Processor Side Signals

Signal	Dir.	Description
VALID	In	Processor informs the master that its request is valid[cite: 1].
MID	In	Processor informs the master that this beat is part of an ongoing INCR burst[cite: 1].
WRITE	In	HIGH for a write request, LOW for a read request[cite: 1].
ADDR[31:0]	In	Address the processor wants to access[cite: 1].
WDATA[31:0]	In	Data the processor wants written[cite: 1].
SIZE[2:0]	In	Size of the requested transfer[cite: 1].
ACCEPT	Out	Master indicates it has taken the request (for reads, this does not mean the data has arrived yet)[cite: 1].
RDATA[31:0]	Out	Data returned from the address that was requested for a read[cite: 1].
RESP	Out	Whether the transfer status is OKAY or ERROR[cite: 1].
RVALID	Out	Master informs the processor that valid read data is available now[cite: 1].

4 Finite State Machine

The master's control logic is implemented as a 4-state Moore-style finite state machine whose current state is driven directly onto HTRANS, matching the AHB-Lite transfer type encoding (IDLE = 2'b00, BUSY = 2'b01, NONSEQ = 2'b10, SEQ = 2'b11)[cite: 1]. The FSM diagram is shown in Figure 4.

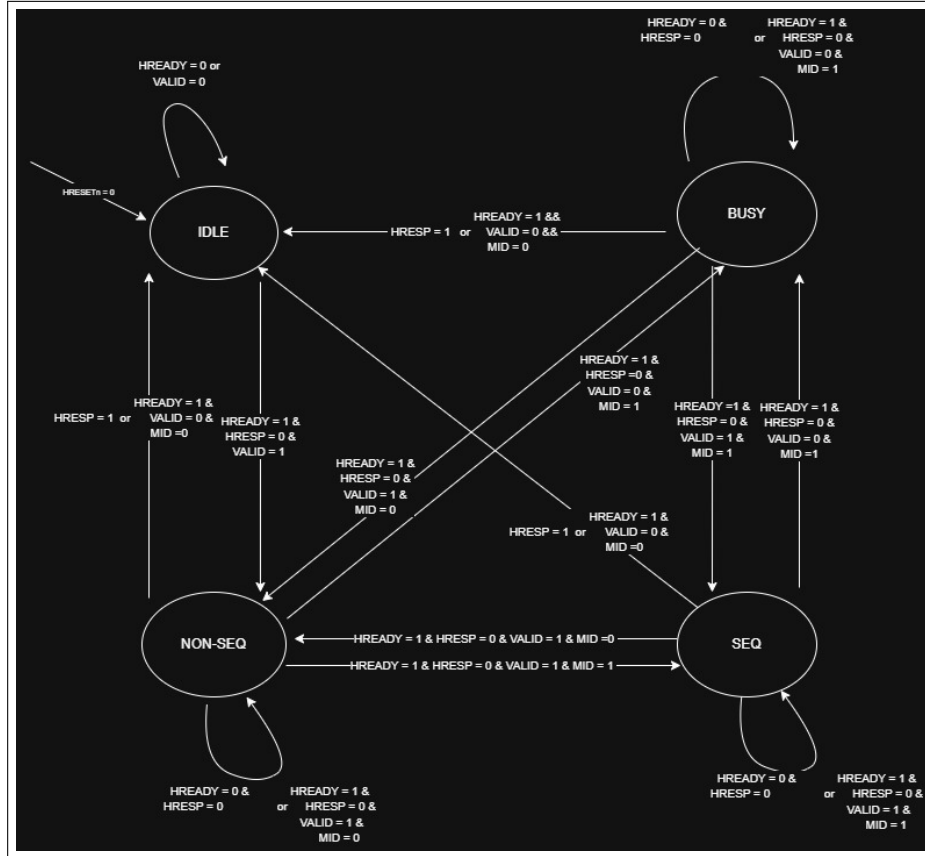


Figure 3: FSM state diagram of the AHB-Lite master.

4.1 States

- **IDLE** – No transfer is being requested[cite: 1]. This is the reset state[cite: 1].
- **BUSY** – The master is continuing a burst but the next beat is not yet ready; address and control signals for the pending beat are held[cite: 1].
- **NONSEQ** – A new, unrelated transfer (single transfer or the first beat of a burst) is in progress[cite: 1].
- **SEQ** – A subsequent beat of an ongoing incrementing burst is in progress[cite: 1].

4.2 Transition Conditions

The next state depends on HREADY, HRESP, VALID and MID[cite: 1]. The general behaviour, common to the BUSY, NONSEQ and SEQ states, can be summarised as follows:

VALID	MID	Meaning / Target State
0	0	Nothing pending → IDLE[cite: 1]
0	1	Burst continues, not ready → BUSY (or stays if already BUSY)[cite: 1]
1	0	New, unrelated request ready → NONSEQ (or stays if already NONSEQ)[cite: 1]
1	1	Burst continues, ready → SEQ (or stays if already SEQ)[cite: 1]

From IDLE:

- Self-loop (stay in IDLE): HREADY=0 or VALID=0[cite: 1].
- IDLE → NONSEQ: HREADY=1 & HRESP=0 & VALID=1[cite: 1].

From BUSY:

- Self-loop: (HREADY=0 & HRESP=0) or (HREADY=1 & HRESP = 0 & VALID = 0 & MID = 1)[cite: 1].
- BUSY → IDLE: HRESP = 1 or (HREADY = 1 & VALID = 0 & MID = 0)[cite: 1].
- BUSY → NONSEQ: HREADY = 1 & HRESP = 0 & VALID = 1 & MID = 0[cite: 1].
- BUSY → SEQ: HREADY = 1 & HRESP = 0 & VALID = 1 & MID = 1[cite: 1].

From NON-SEQ:

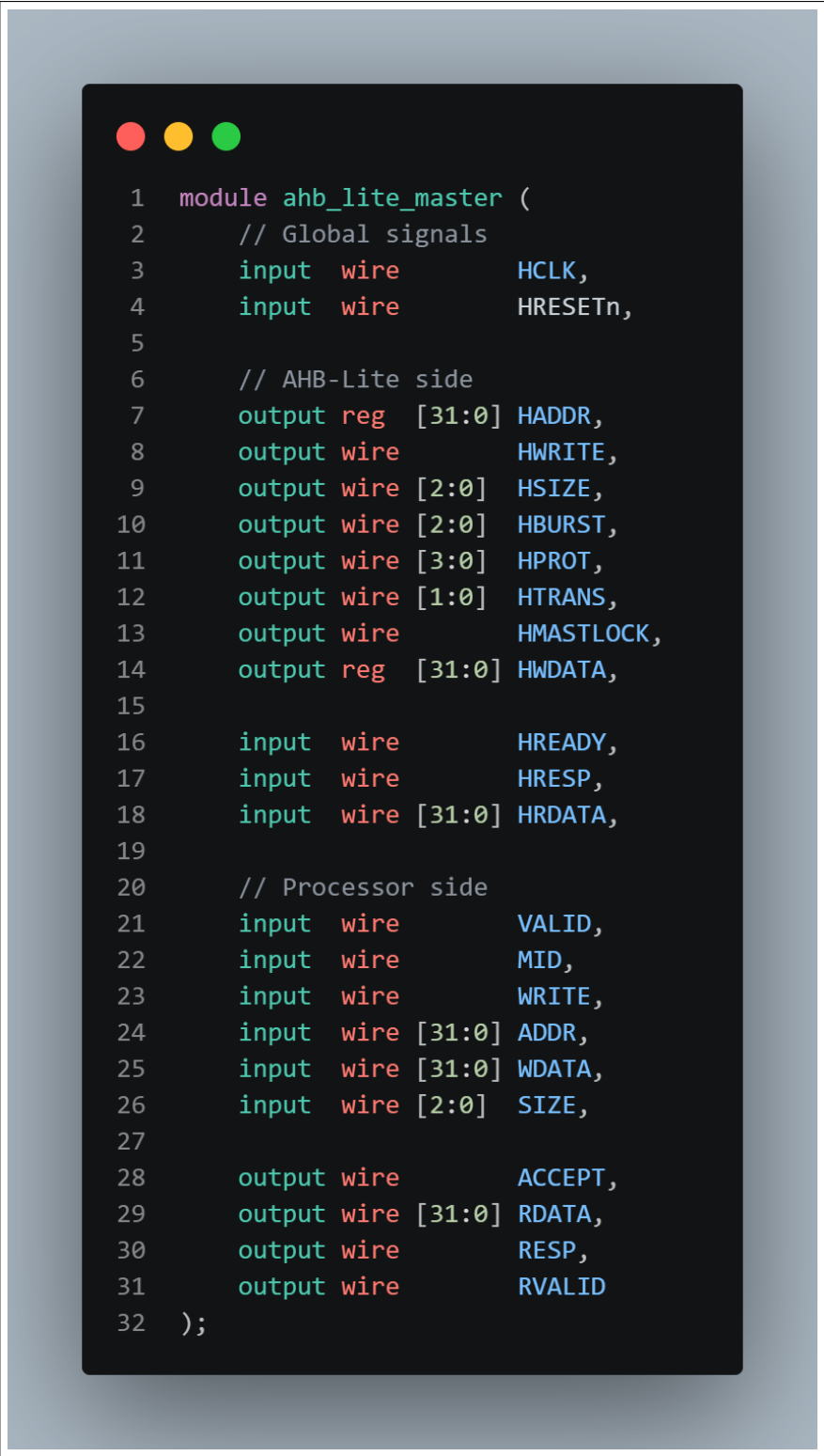
- Self-loop: (HREADY = 0 & HRESP = 0) or (HREADY = 1 & HRESP = 0 & VALID = 1 & MID = 0)[cite: 1].
- NONSEQ → IDLE: HRESP = 1 or (HREADY = 1 & VALID = 0 & MID = 0)[cite: 1].
- NONSEQ → BUSY: HREADY = 1 & HRESP = 0 & VALID = 0 & MID = 1[cite: 1].
- NONSEQ → SEQ: HREADY = 1 & HRESP = 0 & VALID = 1 & MID = 1[cite: 1].

From SEQ:

- Self-loop: (HREADY = 0 & HRESP = 0) or (HREADY = 1 & HRESP = 0 & VALID = 1 & MID = 1)[cite: 1].
- SEQ → IDLE: HRESP = 1 or (HREADY = 1 & VALID = 0 & MID = 0)[cite: 1].
- SEQ → NONSEQ: HREADY = 1 & HRESP = 0 & VALID = 1 & MID = 0[cite: 1].
- SEQ → BUSY: HREADY = 1 & HRESP = 0 & VALID = 0 & MID = 1[cite: 1].

In every non-idle state, an HRESP ERROR response forces an immediate transition back to IDLE, regardless of VALID or MID, reflecting the AHB-Lite requirement that a slave error cancels the remainder of the current transfer sequence[cite: 1]. Whenever HREADY is LOW (and there is no error), the FSM holds its current state so that the address and control signals of the transfer in progress remain stable until the slave is ready[cite: 1].

5 RTL Code



```
1  module ahb_lite_master (  
2      // Global signals  
3      input  wire      HCLK,  
4      input  wire      HRESETn,  
5  
6      // AHB-Lite side  
7      output reg [31:0] HADDR,  
8      output wire      HWRITE,  
9      output wire [2:0] HSIZE,  
10     output wire [2:0] HBURST,  
11     output wire [3:0] HPROT,  
12     output wire [1:0] HTRANS,  
13     output wire      HMASTLOCK,  
14     output reg [31:0] HWDATA,  
15  
16     input  wire      HREADY,  
17     input  wire      HRESP,  
18     input  wire [31:0] HRDATA,  
19  
20     // Processor side  
21     input  wire      VALID,  
22     input  wire      MID,  
23     input  wire      WRITE,  
24     input  wire [31:0] ADDR,  
25     input  wire [31:0] WDATA,  
26     input  wire [2:0] SIZE,  
27  
28     output wire      ACCEPT,  
29     output wire [31:0] RDATA,  
30     output wire      RESP,  
31     output wire      RVALID  
32 );
```

Figure 4: RTL Code – Part 1

```

1  localparam [1:0] ST_IDLE   = 2'b00,
2                      ST_BUSY   = 2'b01,
3                      ST_NONSEQ = 2'b10,
4                      ST_SEQ    = 2'b11;
5
6  reg [1:0] state, next_state;
7
8
9  always @(posedge HCLK or negedge HRESETn) begin
10     if (!HRESETn)
11         state <= ST_IDLE;
12     else
13         state <= next_state;
14 end
15
16
17 always @(*) begin
18     next_state = ST_IDLE;
19     case (state)
20         ST_IDLE: begin
21             if (!HREADY || !VALID)
22                 next_state = ST_IDLE;
23             else
24                 next_state = ST_NONSEQ;
25         end
26
27         ST_BUSY, ST_NONSEQ, ST_SEQ: begin
28             if (HRESP)
29                 next_state = ST_IDLE;
30             else if (!HREADY)
31                 next_state = state;
32             else begin
33                 case ({VALID, MID})
34                     2'b00: next_state = ST_IDLE;
35                     2'b01: next_state = ST_BUSY;
36                     2'b10: next_state = ST_NONSEQ;
37                     2'b11: next_state = ST_SEQ;
38                     default: next_state = ST_IDLE;
39                 endcase
40             end
41         end
42
43         default: next_state = ST_IDLE;
44     endcase
45 end
46

```

Figure 5: RTL Code – Part 2

```

1  assign HTRANS = state;
2
3
4
5  reg fresh_accept, burst_advance;
6
7  always @(*) begin
8      fresh_accept = 1'b0;
9      burst_advance = 1'b0;
10     if (HREADY && !HRESP) begin
11         if (state == ST_IDLE) begin
12             fresh_accept = VALID;
13         end else begin
14             if (MID)
15                 burst_advance = 1'b1;
16             else if (VALID)
17                 fresh_accept = 1'b1;
18         end
19     end
20 end
21
22 reg [31:0] addr_r;
23 reg [2:0] size_r;
24 reg       write_r;
25
26 wire [31:0] incr_bytes = 32'd1 << size_r;
27
28 always @(posedge HCLK or negedge HRESETn) begin
29     if (!HRESETn) begin
30         addr_r <= 32'd0;
31         size_r <= 3'd0;
32         write_r <= 1'b0;
33     end else if (fresh_accept) begin
34         addr_r <= ADDR;
35         size_r <= SIZE;
36         write_r <= WRITE;
37     end else if (burst_advance) begin
38         addr_r <= addr_r + incr_bytes;
39     end
40 end
41

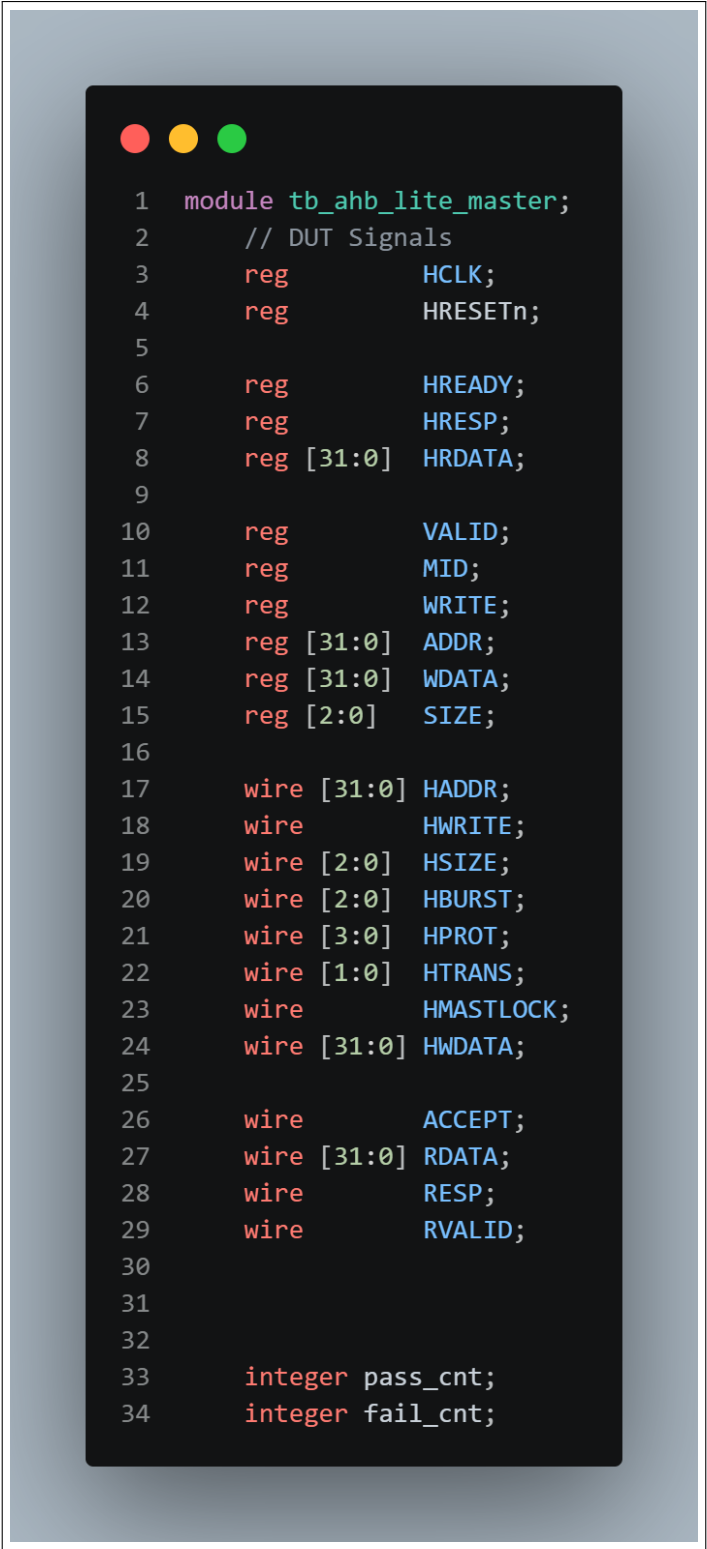
```

Figure 6: RTL Code – Part 3

```
1  always @(*) begin
2      HADDR = addr_r;
3  end
4
5  assign HWRITE = write_r;
6  assign HSIZE = size_r;
7
8  assign ACCEPT = HREADY & ~HRESP & VALID;
9
10 always @(posedge HCLK) begin
11     if (ACCEPT)
12         HWDATA <= WDATA;
13 end
14
15 assign RVALID = (state == ST_NONSEQ || state == ST_SEQ) & HREADY;
16 assign RDATA = HRDATA;
17 assign RESP = HRESP;
18
19 assign HBURST = 3'b001; //for simplicity we will assume that any transfer in INCR and if it is a single burst it will do just one beat
20 assign HPROT = 4'b0011; //unsupported
21 assign HMASTLOCK = 1'b0; //insupported
22
23 endmodule
```

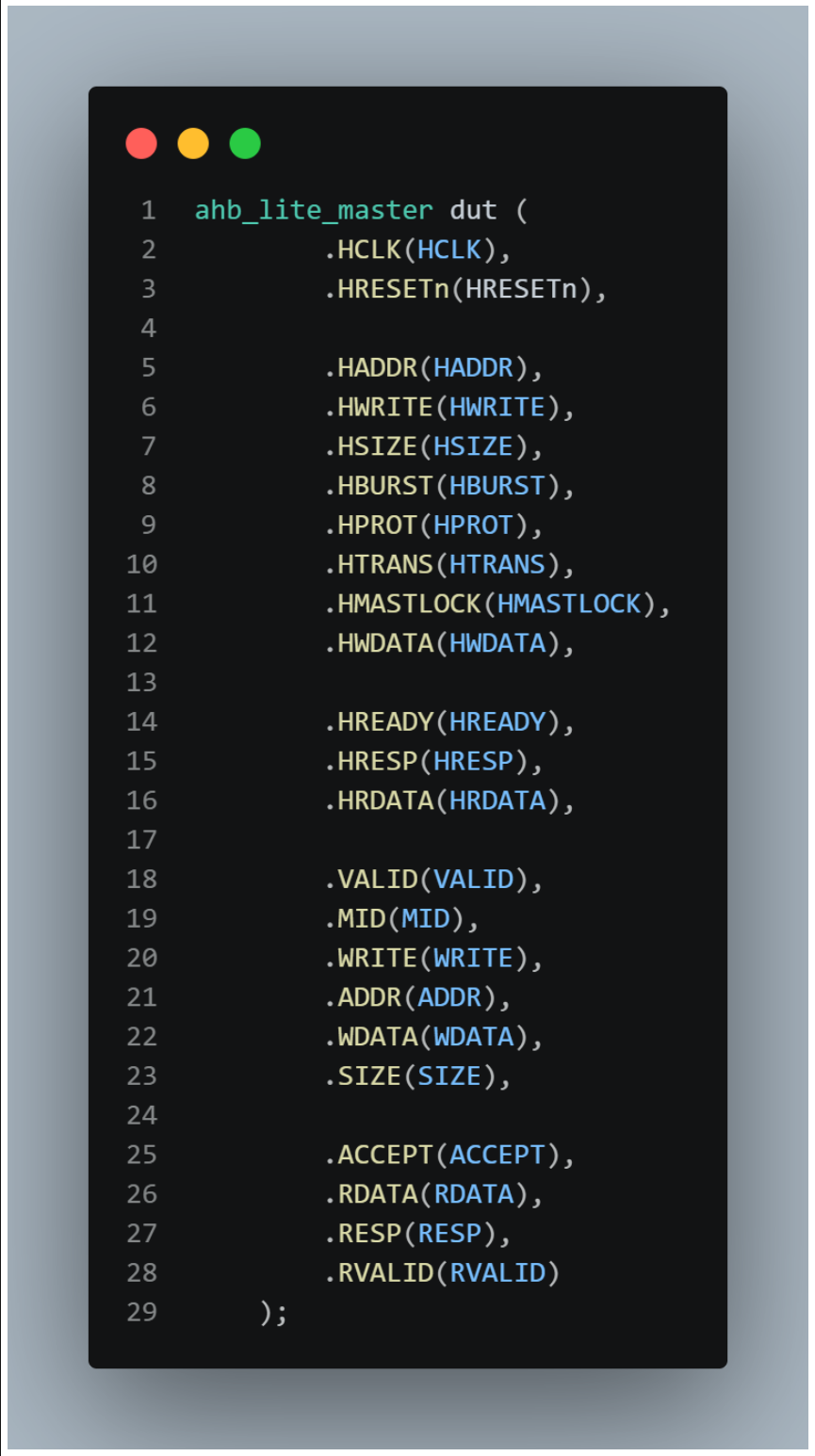
Figure 7: RTL Code – Part 4

6 Testbench Code



```
1 module tb_ahb_lite_master;
2     // DUT Signals
3     reg        HCLK;
4     reg        HRESETn;
5
6     reg        HREADY;
7     reg        HRESP;
8     reg [31:0] HRDATA;
9
10    reg        VALID;
11    reg        MID;
12    reg        WRITE;
13    reg [31:0] ADDR;
14    reg [31:0] WDATA;
15    reg [2:0]   SIZE;
16
17    wire [31:0] HADDR;
18    wire        HWRITE;
19    wire [2:0]   HSIZE;
20    wire [2:0]   HBURST;
21    wire [3:0]   HPROT;
22    wire [1:0]   HTRANS;
23    wire        HMASTLOCK;
24    wire [31:0] HWDATA;
25
26    wire        ACCEPT;
27    wire [31:0] RDATA;
28    wire        RESP;
29    wire        RVALID;
30
31
32
33    integer pass_cnt;
34    integer fail_cnt;
```

Figure 8: Testbench Code – Part 1

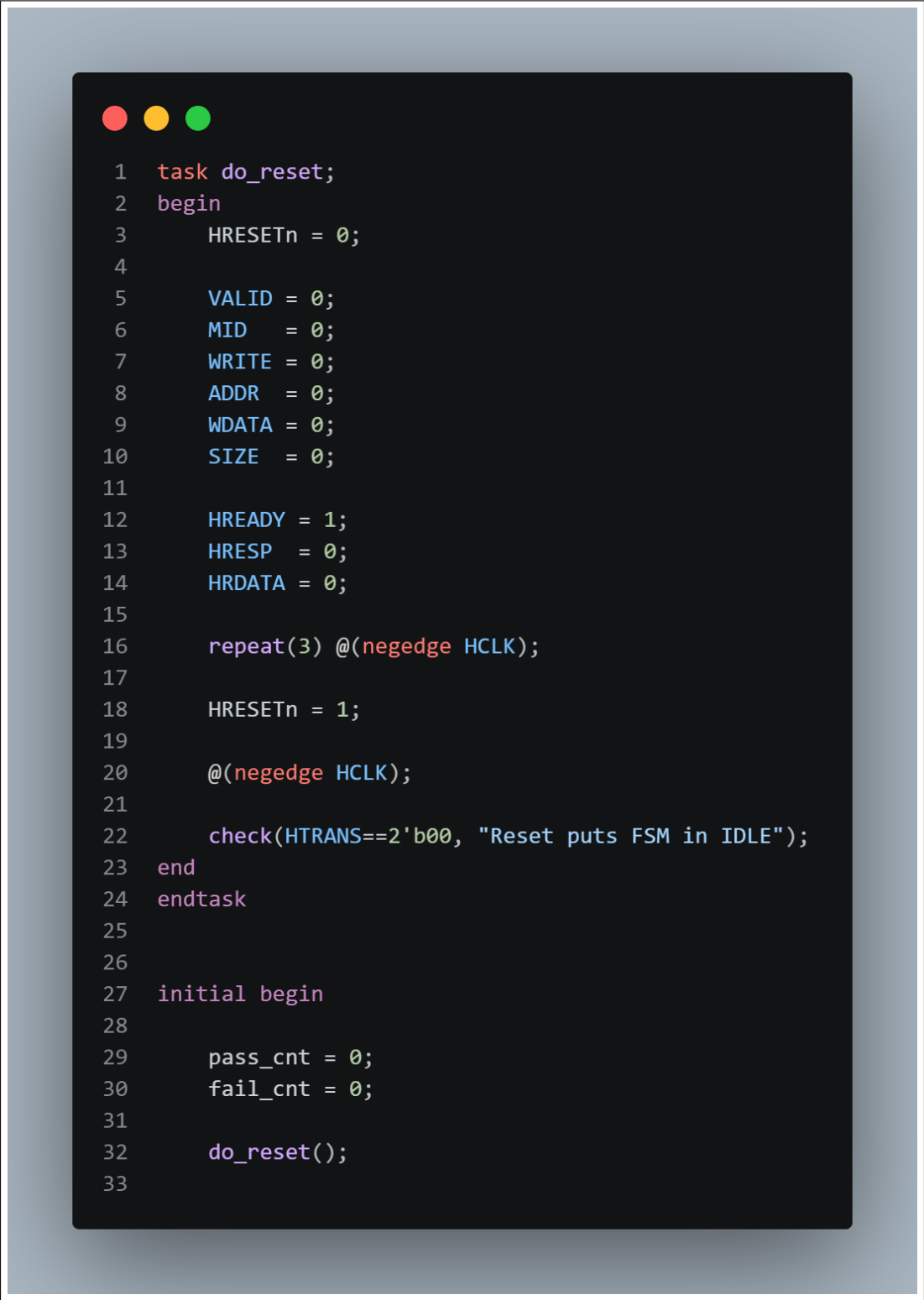


```
1  ahb_lite_master dut (
2      .HCLK(HCLK),
3      .HRESETn(HRESETn),
4
5      .HADDR(HADDR),
6      .HWRITE(HWRITE),
7      .HSIZE(HSIZE),
8      .HBURST(HBURST),
9      .HPROT(HPROT),
10     .HTRANS(HTRANS),
11     .HMASTLOCK(HMASTLOCK),
12     .HWDATA(HWDATA),
13
14     .HREADY(HREADY),
15     .HRESP(HRESP),
16     .HRDATA(HRDATA),
17
18     .VALID(VALID),
19     .MID(MID),
20     .WRITE(WRITE),
21     .ADDR(ADDR),
22     .WDATA(WDATA),
23     .SIZE(SIZE),
24
25     .ACCEPT(ACCEPT),
26     .RDATA(RDATA),
27     .RESP(RESP),
28     .RVALID(RVALID)
29 );
```

Figure 9: Testbench Code – Part 2

```
1  initial begin
2      HCLK = 0;
3      forever #5 HCLK = ~HCLK;
4  end
5
6
7
8  task check;
9      input condition;
10     input string msg;
11     begin
12         $display("time = %0t", $time);
13         if(condition) begin
14             pass_cnt = pass_cnt + 1;
15             $display("PASS");
16         end
17         else begin
18             fail_cnt = fail_cnt + 1;
19             $display("FAIL");
20         end
21         $display("%s",msg);
22     end
23 endtask
24
25
26
27 task send_req;
28     input wr;
29     input mid_i;
30     input [31:0] addr_i;
31     input [31:0] data_i;
32     input [2:0] size_i;
33
34     begin
35         VALID <= 1;
36         MID   <= mid_i;
37         WRITE <= wr;
38         ADDR  <= addr_i;
39         WDATA <= data_i;
40         SIZE  <= size_i;
41
42         @(negedge HCLK);
43     end
44 endtask
```

Figure 10: Testbench Code – Part 3



```
1  task do_reset;
2  begin
3      HRESETn = 0;
4
5      VALID = 0;
6      MID   = 0;
7      WRITE = 0;
8      ADDR  = 0;
9      WDATA = 0;
10     SIZE  = 0;
11
12     HREADY = 1;
13     HRESP  = 0;
14     HRDATA = 0;
15
16     repeat(3) @(negedge HCLK);
17
18     HRESETn = 1;
19
20     @(negedge HCLK);
21
22     check(HTRANS==2'b00, "Reset puts FSM in IDLE");
23 end
24 endtask
25
26
27 initial begin
28
29     pass_cnt = 0;
30     fail_cnt = 0;
31
32     do_reset();
33
```

Figure 11: Testbench Code – Part 4

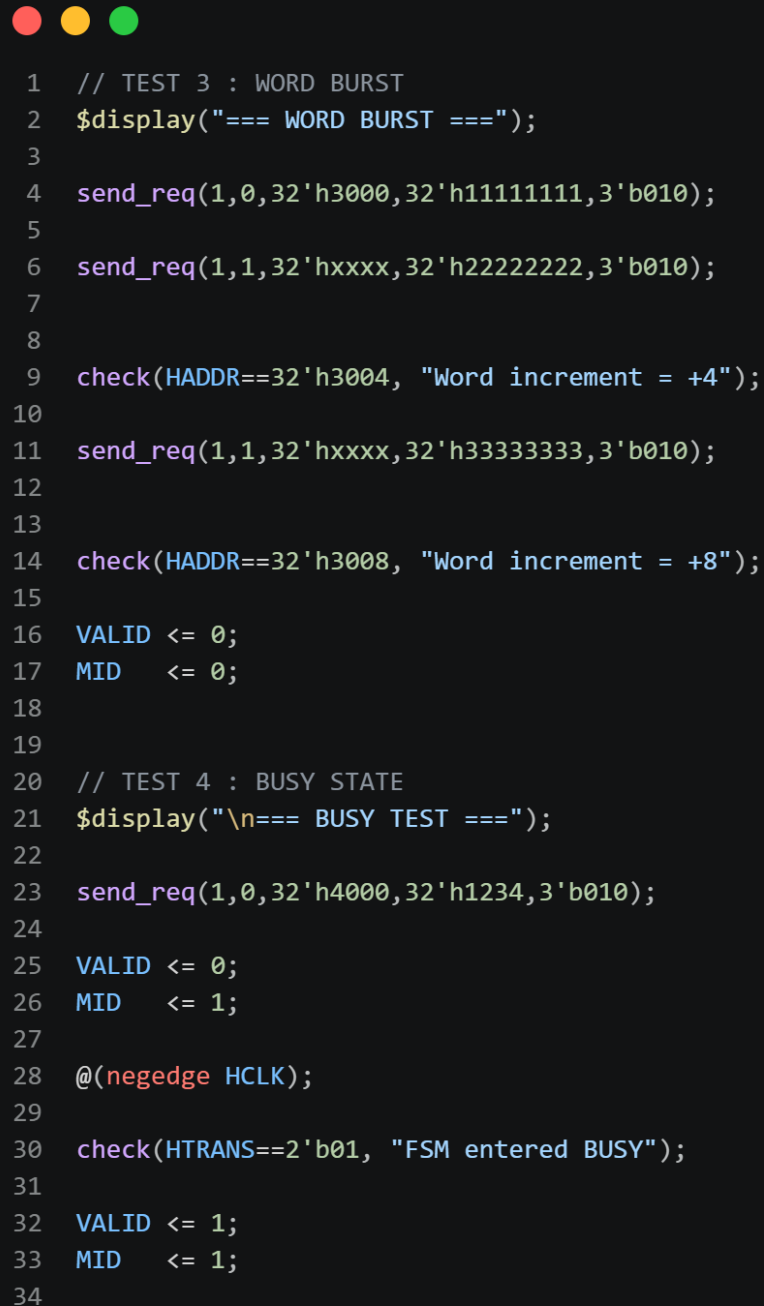


```

1  // TEST 1 : BYTE BURST
2  $display("=== BYTE BURST ===");
3
4  send_req(1,0,32'h1000,32'h11,3'b000);
5
6  check(ACCEPT,"BYTE burst first beat accepted");
7
8
9  send_req(1,1,32'hxxxx,32'h22,3'b000);
10
11 check(HADDR==32'h1001, "Byte increment = 1");
12
13
14 send_req(1,1,32'hxxxx,32'h33,3'b000);
15
16 check(HADDR==32'h1002, "Byte increment = 2");
17
18 @(negedge HCLK);
19
20
21 // TEST 2 : HALFWORD BURST
22 $display("=== HALFWORD BURST ===");
23
24 send_req(1,0,32'h2000,32'hAAAA,3'b001);
25
26 send_req(1,1,32'hxxxx,32'hBBBB,3'b001);
27
28
29 check(HADDR==32'h2002, "Halfword increment = +2");
30
31 send_req(1,1,32'hxxxx,32'hCCCC,3'b001);
32
33
34 check(HADDR==32'h2004, "Halfword increment = +4");
35
36 VALID <= 0;
37 MID   <= 0;
38
39 @(negedge HCLK);
40

```

Figure 12: Testbench Code – Part 5



```
1 // TEST 3 : WORD BURST
2 $display("=== WORD BURST ===");
3
4 send_req(1,0,32'h3000,32'h11111111,3'b010);
5
6 send_req(1,1,32'hxxxx,32'h22222222,3'b010);
7
8
9 check(HADDR==32'h3004, "Word increment = +4");
10
11 send_req(1,1,32'hxxxx,32'h33333333,3'b010);
12
13
14 check(HADDR==32'h3008, "Word increment = +8");
15
16 VALID <= 0;
17 MID    <= 0;
18
19
20 // TEST 4 : BUSY STATE
21 $display("\n=== BUSY TEST ===");
22
23 send_req(1,0,32'h4000,32'h1234,3'b010);
24
25 VALID <= 0;
26 MID    <= 1;
27
28 @(negedge HCLK);
29
30 check(HTRANS==2'b01, "FSM entered BUSY");
31
32 VALID <= 1;
33 MID    <= 1;
34
```

Figure 13: Testbench Code – Part 6

```
1  @(negedge HCLK);
2
3  check(HTRANS==2'b11, "BUSY -> SEQ");
4
5  VALID <= 0;
6  MID   <= 0;
7
8
9  // TEST 5 : WAIT STATES
10 $display("\n=== WAIT STATE TEST ===");
11
12 send_req(1,0,32'h5000,32'hDEADBEEF,3'b010);
13
14 HREADY <= 0;
15
16 repeat(3) begin
17     @(negedge HCLK);
18
19     check(HTRANS==2'b10, "State held during wait");
20 end
21
22 HREADY <= 1;
23
24 @(negedge HCLK);
25
26 VALID <= 0;
27
28
29 // TEST 6 : READ DATA RETURN
30 $display("\n=== READ TEST ===");
31
32 send_req(0,0,32'h6000,32'h0,3'b010);
33
34 HRDATA <= 32'hCAFEBABE;
35
```

Figure 14: Testbench Code – Part 7



```

1      @(negedge HCLK);
2
3      check(RVALID==1, "RVALID asserted");
4
5      check(RDATA==32'hCAFEBABE, "Read data matched");
6
7      VALID <= 0;
8
9
10     // TEST 7 : ERROR RESPONSE
11     $display("\n=== ERROR TEST ===");
12
13     send_req(0,0,32'h7000,32'h0,3'b010);
14
15     HRESP <= 1;
16
17     @(negedge HCLK);
18
19     check(RESF==1, "ERROR response propagated");
20
21
22     check(HTRANS==2'b00,
23           "FSM returned to IDLE after error");
24
25
26
27     repeat(5) @(negedge HCLK);
28
29     $display("=====");
30     $display("PASSED =
31 ", pass_cnt);
31     $display("FAILED =
32 ", fail_cnt);
32
33
34     $finish;
35
36     end
37
38 endmodule

```

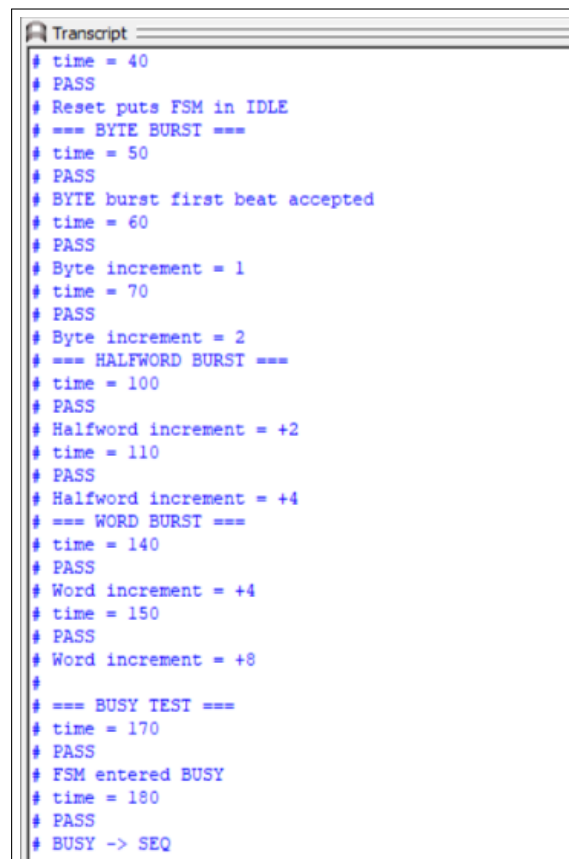
Figure 15: Testbench Code – Part 8

7 Simulation Results

This section presents the verification logs and waveform captures obtained from running the test-bench. The verification environment tests byte, halfword, and word burst transfers, insertion of master busy states, slave wait-state stalls, valid read-data return, and slave error handling[cite: 2].

7.1 Terminal Execution Logs

The simulation transcript confirms successful assertion checking across all test cases without any failure assertions[cite: 2].



```
# time = 40
# PASS
# Reset puts FSM in IDLE
# === BYTE BURST ===
# time = 50
# PASS
# BYTE burst first beat accepted
# time = 60
# PASS
# Byte increment = 1
# time = 70
# PASS
# Byte increment = 2
# === HALFWORD BURST ===
# time = 100
# PASS
# Halfword increment = +2
# time = 110
# PASS
# Halfword increment = +4
# === WORD BURST ===
# time = 140
# PASS
# Word increment = +4
# time = 150
# PASS
# Word increment = +8
#
# === BUSY TEST ===
# time = 170
# PASS
# FSM entered BUSY
# time = 180
# PASS
# BUSY -> SEQ
```

Figure 16: Simulation Terminal Output – Log Part 1

```

#
# === WAIT STATE TEST ===
# time = 200
# PASS
# State held during wait
# time = 210
# PASS
# State held during wait
# time = 220
# PASS
# State held during wait
#
# === READ TEST ===
# time = 250
# PASS
# RVALID asserted
# time = 250
# PASS
# Read data matched
#
# === ERROR TEST ===
# time = 270
# PASS
# ERROR response propagated
# time = 270
# PASS
# FSM returned to IDLE after error
# =====
# PASSED = 17
# FAILED = 0
# ** Note: $finish      : E:/ADI_intern/Assignments/AHB_master/Test.sv(292)
#      Time: 320 ns   Iteration: 1   Instance: /tb_ahb_lite_master

```

Figure 17: Simulation Terminal Output – Log Part 2

7.2 Test 1: Byte Burst Transfer

This test validates byte-wide transfers (SIZE = 3'b000) in an incrementing burst[cite: 2]. The initial request starts at base address 0x1000[cite: 2]. With each subsequent burst beat indicated by MID = 1, the internal address generator increments HADDR by +1 byte (0x1000 → 0x1001 → 0x1002).

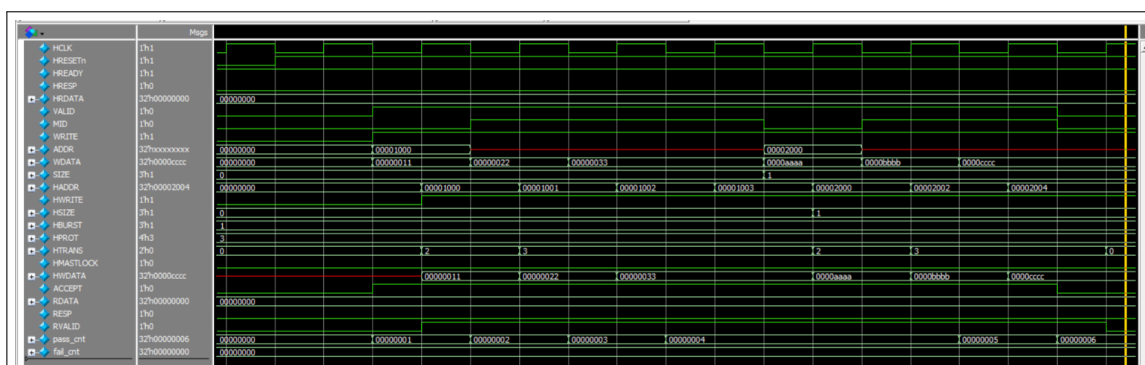


Figure 18: Waveform: Byte Burst Transfer (SIZE = 000)

7.3 Test 2: Halfword Burst Transfer

This test verifies 16-bit halfword transfers (SIZE = 3'b001) initiating at base address 0x2000[cite: 2]. As sequential burst transfers advance, HADDR correctly increments by +2 bytes per clock cycle (0x2000 → 0x2002 → 0x2004).

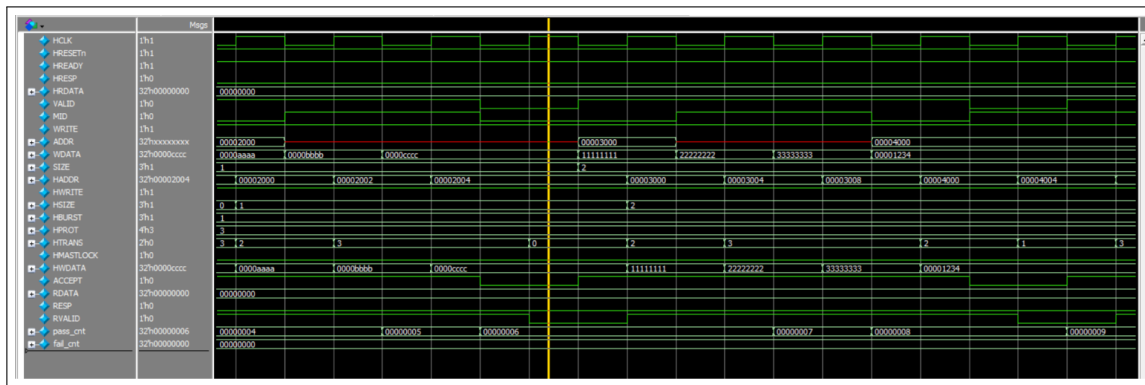


Figure 19: Waveform: Halfword Burst Transfer (SIZE = 001)

7.4 Test 3: Word Burst Transfer

This test evaluates 32-bit word transfers (SIZE = 3'b010) starting from base address 0x3000[cite: 2]. The internal address computation updates HADDR by +4 bytes on each sequential beat (0x3000 → 0x3004 → 0x3008).

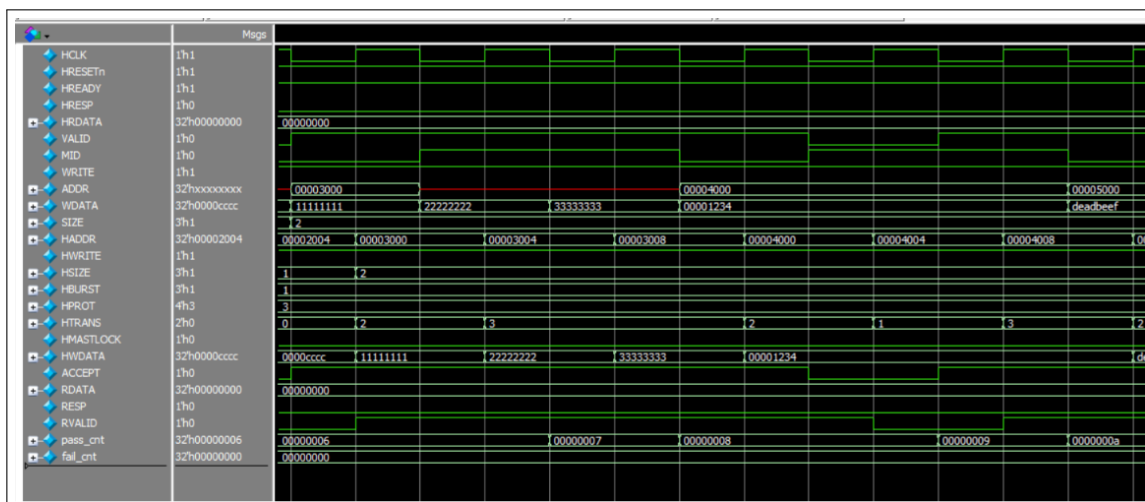


Figure 20: Waveform: Word Burst Transfer (SIZE = 010)

7.5 Test 4: Master Busy State Handling

This test checks the master's ability to insert an intermediate busy state during a burst access[cite: 2]. When the processor deasserts VALID while keeping MID = 1, the master FSM transitions into BUSY (HTRANS = 2'b01) and holds current address/control lines[cite: 2]. Once VALID returns HIGH with MID = 1, the FSM seamlessly resumes into the SEQ state (HTRANS = 2'b11).

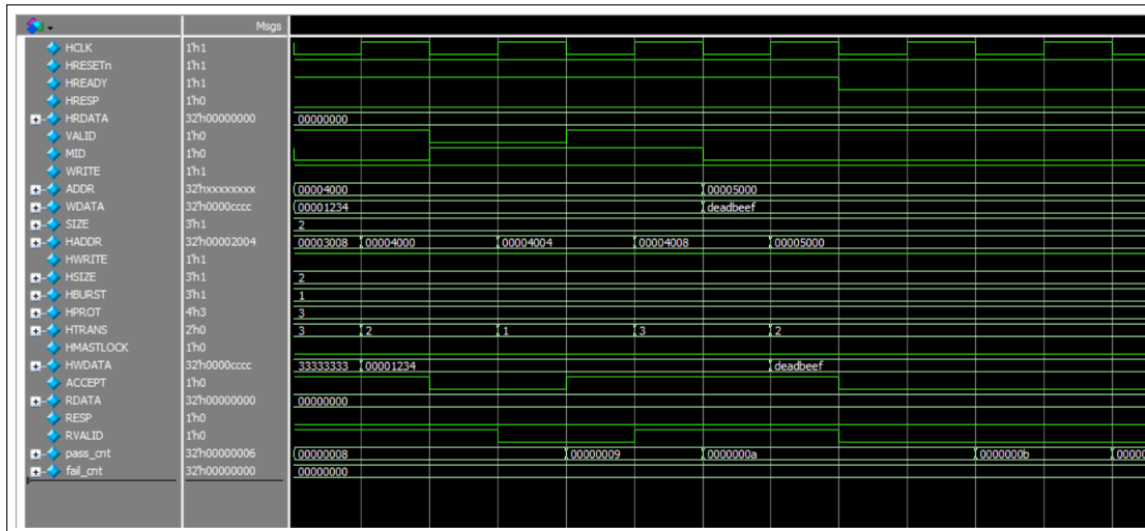


Figure 21: Waveform: Master Insertion of BUSY State

7.6 Test 5: Slave Wait-State Handling

This test demonstrates master behavior during slave wait states[cite: 2]. When HREADY is driven LOW by a slow slave, the master FSM freezes its current state (HTRANS = 2'b10) and maintains stable output address/control signals for multiple cycles until HREADY is re-asserted.

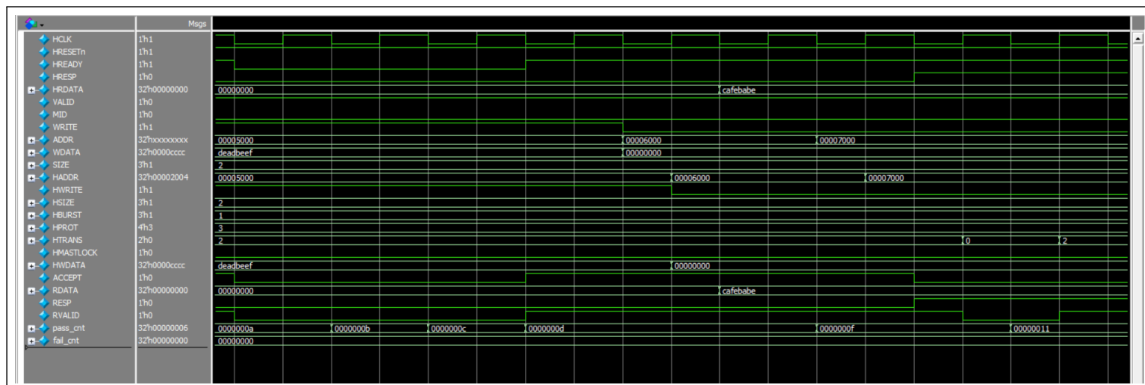


Figure 22: Waveform: Slave Wait-States (HREADY = 0)

7.7 Test 6: Read Data Return

This test verifies a single read transaction (WRITE = 0) to address 0x6000[cite: 2]. The master asserts RVALID when valid data is presented on HRDATA during an active state, capturing and routing 0xCAFEBABE directly to RDATA.

7.8 Test 7: Slave Error Response Handling

This test checks protocol handling during a slave error[cite: 2]. When HRESP is asserted HIGH by the slave in response to a request at address 0x7000, the master immediately propagates the error via RESP = 1 and forces an abort back to IDLE state (HTRANS = 2'b00).

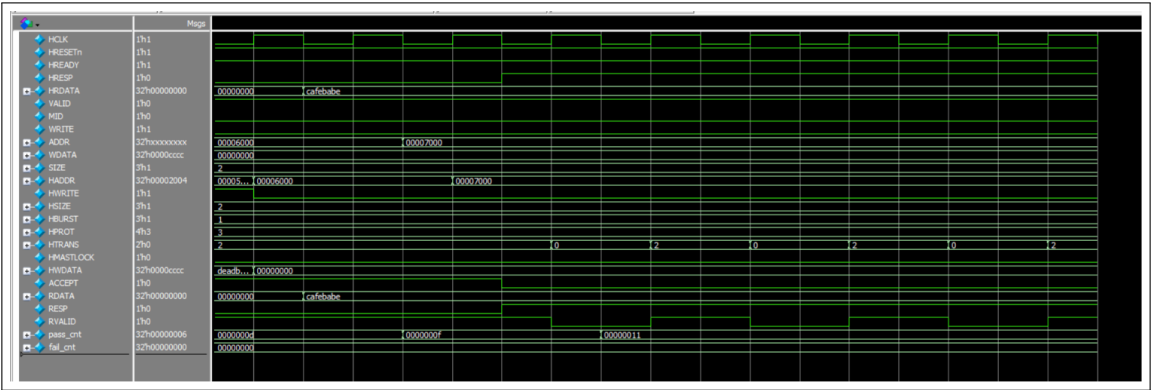


Figure 23: Waveform: Read Transfer and Error Handling